

Build your own Summit robot

For teams without the KitBot. Design your own chassis and strategy around a micro:bit. Start with the custom template below so the tournament controls stay separate from your work.

Custom Robot Tournament Template v2.0

Empty strategy plus empty hardware adapters. No Wukong, light-strip or sensor calls are built in. Choose Edit Code and rename your copy.

<https://makecode.microbit.org/S27115-01102-79267-06002>



This is a starting framework, not a ready-to-drive program

It can receive tournament commands, but cannot control your motors until you add the correct motor-board extension and complete the hardware adapters. A checkmark means the match started; it does not verify motor wiring.

Plan the robot before connecting power

Plan	Practical advice
Chassis and drive	Use a compact, stiff frame. Keep heavy parts low and secure. Check wheel traction, ground clearance and climbing behavior on the actual field.
Controller and motor board	Use one micro:bit for decisions. Choose a compatible motor driver; record its model, extension and motor ports. Do not connect drive motors directly to micro:bit pins.
Sensors	Start with a reliable edge sensor. Mount it so it sees the boundary before the wheels reach the drop. Add opponent sensing after basic movement and recovery work.
Power and wiring	Use each device within its voltage/current specifications. Confirm signal-voltage compatibility and ground connections. Keep a tool-free drive-power cutoff accessible.

Check the current game manual

The supplied v1.0 manual specifies a 10 × 10 cm starting footprint, 500 g maximum including the battery, and one micro:bit making decisions. It also covers the power cutoff, permitted mechanisms and visible match color. Confirm current requirements before building; this guide is not the rulebook.

[Open the current SRC game manual](#) | [micro:bit power specifications](#)

Connect your hardware

The workspace has three areas: your strategy on the left, hardware adapters in the middle, and folded tournament controls on the right. An **adapter** is a small function that translates a request into blocks for your particular board.

1. Set **myTeam** in on start to your assigned team number. Keep the call to `TOURNAMENT_setup`.
2. Add the MakeCode extension recommended by your motor-board manufacturer. Confirm its setup, motor direction, speed range and stop commands from its documentation.
3. Complete the three **HARDWARE** functions below. Keep them short; do not add pauses or forever loops. No motor command belongs in the color-display function.
4. With wheels raised, test motor directions at a low setting. Make **positive left/right inputs mean forward on each side** in your adapter. Then strategy code can use equal positive inputs to drive forward.

Your hardware function	Required behavior
<code>HARDWARE_setup</code>	Initialize the motor board and sensors once at boot, with outputs stopped. Tournament setup calls it before issuing the first stop command.
<code>HARDWARE_setMotors(left, right)</code>	Set BOTH motors using the supplied inputs. Map signs/scales to your board. Inputs 0, 0 MUST stop both outputs immediately. Tournament start/stop protection depends on this.
<code>HARDWARE_showMatchColor(color)</code>	If using lights, display RED or BLUE when requested; WAIT is the idle indication. Keep this non-blocking. If using a manual marker, leave it empty and follow the referee schedule for the marker.

Keep the tournament logic unchanged

Leave **TOURNAMENT** functions, button/radio events and the forever loop alone. Your strategy must call **TOURNAMENT_drive**, which checks whether movement is allowed. Do not call `HARDWARE_setMotors` directly from your strategy; that would bypass start/stop protection.

Keep a pin map

Record the board model and extension, both motor ports, each sensor pin, power connections, and the match indicator. Check for shared-pin conflicts before connecting power.

Write a small strategy, then test it

Put your repeated sensor checks and decisions inside **YOUR_robotCode**. The supplied loop calls it only during an active match. It starts empty and provides no edge protection.

Start with four decisions

Invalid floor reading → stop.

Edge detected or recovery active → recover.

Valid nearby opponent → approach.

Otherwise → search slowly.

Make recovery measurable

Use an elapsed-time variable to reverse briefly, then turn and recheck. Keep sampling the edge while turning. Tune on your robot: timed turns are not guaranteed angles, and a single front sensor cannot see a rear edge. Avoid long pauses between checks.

Test in this order	What counts as a pass
1 / Stationary sensor readings	Black is distinguishable from every field color, including tilt and shadows. A missing reading is handled deliberately.
2 / Raised-wheel movement	Left/right direction is correct. A low-speed forward command and a 0, 0 command work. No motion occurs at boot.
3 / Start and stop	A waits five seconds. B cancels the countdown or stops motion; the robot stays stopped until reset. Test during every behavior.
4 / Actual referee controller	Correct team number, color/marker, addressed START and STOP. Repeat at field distance. Three-minute local cutoff works.
5 / Low-speed field trials	Straight and angled edge approaches recover. A nearby opponent never overrides edge recovery. Retest after changing mounting or speed.

Before the next match

After referee STOP, wait for its X to return to a small diamond, then reset both robots. If only one starts, stop both and replay. Do not start one late. Keep the drive-power cutoff reachable throughout testing.

Make the next change explainable

For each trial, record: your prediction → one change → observed result → keep or undo, and why. Save a working copy before a major change.

Use the sensor labs as examples

These optional projects were written for the KitBot electronics. They teach useful ideas, but they are not universal wiring diagrams or ready-to-run firmware for a different motor board.

Color Lab v2.0

BLACK compares brightness with a threshold. COLORS reports channel values. Wukong stop commands, P16 NeoPixels and the kit TCS34725 extension are assumed.

<https://makecode.microbit.org/S51417-07093-46392-81607>



Ultrasonic Lab v2.0

SENSOR reports distance. MOVE adds a near-target response after A. Assumes Wukong motors and a compatible trigger/echo sensor on P1/P2.

<https://makecode.microbit.org/S29595-96351-49856-35763>



If your hardware differs	Use the idea this way
Different floor sensor	Read it with its own extension. Record its output over every field color at normal height and tilt. Do not copy the KitBot thresholds or channel rules without measuring.
Different distance sensor	Use its supported reading block and units. Identify how it signals no target or an invalid reading. A failed measurement must not cause a charge.
Different motor board	Use the custom template and your completed motor adapter. Copy the decision pattern, not the Wukong motor calls.
No sensors yet	Test the tournament controls with wheels raised. Add reliable edge detection before allowing autonomous field driving.

Build one reliable behavior at a time

Measure first, make one decision, test the response, then combine behaviors. Be able to explain the sensor reading, the motor command, and why STOP prevents further motion.